

Pointers and Memory Layout

A running program's memory is divided into distinct regions. Understanding where variables live is the key to mastering pointers.

```
{ "type": "graph", "nodes": [ [ "Stack (local vars)", 100, 40 ], [ "Heap (dynamic)", 300, 40 ], [ "BSS", 400, 40 ] ] }
```

The Stack grows downward (function frames). The Heap grows upward (malloc/new). Pointers let us reference memory across these regions.

****Pointer Declaration + Dereferencing****

`int *p;` → declares a pointer to int (stores an address)

`p = &x;` → `&` is the "address-of" operator

`*p = 10;` → `*` is the "dereference" operator (reads/writes the value at that address)

****Example: `int x = 5, int *p = &x;`****

0x1000	x = 5
0x1008	= 0x1000

`p` lives at address `0x1008` and holds the value `0x1000` - which is the address of `x`. Dereferencing `*p` follows the arrow to `x` and gives us `5`. Writing `*p = 10` changes `x` to `10`.

****Pointer Arithmetic****

Adding to a pointer advances it by `sizeof(type)` bytes, not 1 byte. The pointer "knows" the size of what it points to.

```
int arr[4] = {10, 20, 30, 40};
```

```
int *ptr = arr; → ptr points to arr[0]
```

```
ptr + 1 → points to arr[1] (advances + bytes for int)
```

```
ptr + 2 → points to arr[2]
```

10	20	30	40
0	1	2	3

`*(ptr + 3)` gives `arr[3] = 40`. This is exactly what `arr[i]` does under the hood!

****Common Pitfalls****

1. Null Pointers - Dereferencing NULL crashes the program.
`int *q = NULL; *q = 5; → works fine SEGFAULT!`

Always check: `if (p != NULL) { *p = 5; }`

2. Dangling Pointers - Pointing to freed memory.

```
int *r = malloc(sizeof(int)),
```

```
free(r);
```

```
*r = 10, → safe to use use-after-free!
```

Set `r = NULL` after `free` to catch accidental dereferences.

3. Uninitialized Pointers - Reading garbage addresses.

```
int *s; *s = 5; →
```

~~no problem~~ writes to a random memory location!

Always initialize: `int *s = NULL`, or `int *s = +valid-var;`

A pointer stores an address - think of it as an arrow